CMSC132: Enumerator Type (Enums)

Chapter 3

By Naren Ayinala

May 2026

Contents

1 What Is an Enum? **2**

1.1 Declaring and Printing Enums 2

2 Built-In Methods: values(), ordinal(), and name() **2**

3 Creating Variables, Fields, and Constructors **2**

4 Class Behavior and Per-Constant Behavior **3**

5 Using Enums with switch **3**

6 Enum Collections and Common Mistakes **3**

6.1 Practice 3

1 What Is an Enum?

An **enumeration** is a type with a fixed set of named instances. Use it when values come from a known list, such as weekdays, directions, or order statuses. Unlike plain string constants, enums are type-safe: the compiler prevents assigning an unrelated string. Enum constants are objects, not integers.

1.1 Declaring and Printing Enums

```
enum Direction { NORTH, EAST, SOUTH, WEST }
class Demo {
    public static void main(String[] args) {
        Direction d = Direction.NORTH;
        System.out.println(d); // NORTH (toString by default)
        // d = "NORTH"; // compile error
    }
}
```

The convention is uppercase enum constant names, although it is not required by Java. An enum can be declared at the top level or nested in a class; it cannot be directly instantiated using `new`.

2 Built-In Methods: values(), ordinal(), and name()

The compiler generates `values()` to return an array of constants in declaration order. The `name()` method returns the exact declared identifier, while `ordinal()` returns the zero-based position. `toString()` initially behaves like `name()`, but can be overridden.

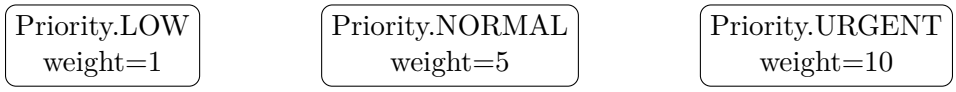
```
enum Level { LOW, MEDIUM, HIGH }
for (Level level : Level.values()) {
    System.out.println(level.name() + " -> " + level.ordinal());
}
// LOW -> 0
// MEDIUM -> 1
// HIGH -> 2
Level x = Level.valueOf("HIGH"); // returns HIGH
```

Important: Avoid storing an enum's ordinal as a persistent ID. Reordering constants changes ordinals, which could corrupt stored meanings. Prefer explicit fields.

3 Creating Variables, Fields, and Constructors

An enum can have private fields, a constructor, and methods. Each constant calls the enum constructor once when the enum class is initialized. The constructor cannot be public or protected; enum constructors are implicitly private.

```
enum Priority {
    LOW(1), NORMAL(5), URGENT(10);
    private final int weight;
    Priority(int weight) { this.weight = weight; }
    public int weight() { return weight; }
}
System.out.println(Priority.URGENT.weight()); // 10
```



4 Class Behavior and Per-Constant Behavior

An enum is a special class extending `java.lang.Enum` internally. It may implement interfaces, define instance methods, and even override a method separately for each constant. It cannot extend another class. You can compare enum constants with `==` because each constant is a unique singleton instance.

```
enum Operation {
    ADD { int apply(int a, int b) { return a + b; } },
    MULTIPLY { int apply(int a, int b) { return a * b; } };
    abstract int apply(int a, int b);
}
System.out.println(Operation.ADD.apply(2, 3)); // 5
System.out.println(Operation.MULTIPLY.apply(2, 3)); // 6
```

5 Using Enums with switch

Enums are ideal switch selectors. In a switch over an enum, write unqualified constant labels such as `case RED:`, not `case Color.RED:`. The older switch statement needs `break` to prevent fall-through. Newer switch expressions use arrows and can return a value.

```
enum Signal { RED, YELLOW, GREEN }
static String action(Signal s) {
    switch (s) {
        case RED: return "Stop";
        case YELLOW: return "Slow down";
        case GREEN: return "Go";
        default: throw new AssertionError(s);
    }
}
```

6 Enum Collections and Common Mistakes

`EnumSet` efficiently represents sets of enum constants; `EnumMap` efficiently maps enum keys to values. Avoid using strings to represent fixed categories, avoid depending on ordinal numbers for business meaning, and do not assume `valueOf` ignores case: it is case-sensitive.

```
import java.util.*;
EnumSet<Signal> caution = EnumSet.of(Signal.RED, Signal.YELLOW);
EnumMap<Signal, Integer> seconds = new EnumMap<>(Signal.class);
seconds.put(Signal.RED, 30);
System.out.println(seconds.get(Signal.RED)); // 30
```

6.1 Practice

1. What does `Level.MEDIUM.ordinal()` return? 1. **2.** Can `new Level()` compile? No. **3.** Why is `==` acceptable for enum constants? Each declared constant is a unique instance. **4.** What happens for

`Level.valueOf("medium")`? An `IllegalArgumentException` because names are case-sensitive.